

Towards a Mathematical Understanding of Neural Network-Based Machine Learning

What we know and what we don't

Alberto Taddei Leon Ackermann

Sorbonne Université – EUMaster4HPC

Paper discussed

Towards a Mathematical Understanding of Neural Network-Based Machine Learning:
what we know and what we don't [1]

Weinan E, Chao Ma, Stephan Wojtowytsch, Lei Wu

December 9, 2020

CSIAM Transactions on Applied Mathematics

Our focus

Approximation, generalization, and training dynamics from a numerical-analysis viewpoint.

Motivation: powerful but fragile

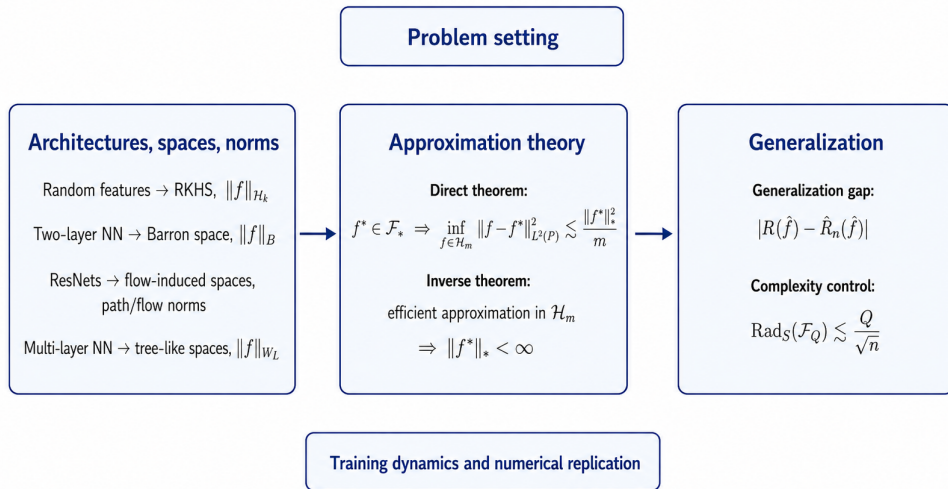
Neural networks are extremely successful in high-dimensional problems, but their behaviour is still difficult to explain mathematically.

- They can approximate very complex functions.
- Their performance depends strongly on architecture, initialization, optimizer and hyperparameters.
- The paper asks for a mathematical explanation of both the success and the fragility.

Main question

Can we explain when neural networks

approximate well, generalize well, and can actually be trained?



Problem setting

Model problem: supervised regression

The paper mainly studies supervised learning as an L^2 -regression problem.

We observe data

$$S = \{(x_i, y_i)\}_{i=1}^n, \quad y_i = f^*(x_i),$$

where

$$x_i \in X = [0, 1]^d, \quad x_i \sim P.$$

- f^* : unknown target function.
- P : input data distribution.
- H_m : hypothesis space, e.g. neural networks of width/size m .
- $\hat{f}$: trained model obtained from finite data.

Population risk, empirical risk and best-in-class model

The true objective is the population risk:

$$R(f) = \mathbb{E}_{x \sim P} [(f(x) - f^*(x))^2] = \|f - f^*\|_{L^2(P)}^2.$$

Training uses the empirical risk:

$$\hat{R}_n(f) = \frac{1}{n} \sum_{i=1}^n (f(x_i) - f^*(x_i))^2.$$

The best function in the hypothesis space is

$$f_m = \arg \min_{f \in H_m} R(f).$$

Theoretical benchmark

$$f_m$$

Best possible function in H_m .

Trained model

$$\hat{f}$$

Training function on finite data.

Approximation and estimation errors

The total error can be decomposed as

$$f^* - \hat{f} = (f^* - f_m) + (f_m - \hat{f}).$$

Approximation error

$$f^* - f_m$$

Due to the expressive power of the hypothesis space H_m .

Estimation error

$$f_m - \hat{f}$$

Due to finite data and the training algorithm.

Goal

The theory should explain approximation, estimation and optimization.

Why high dimension is hard

Classical approximation estimates often look like

$$\|f - f_m\|_{L^2(X)} \leq C m^{-\alpha/d} \|f\|_{H^\alpha(X)}.$$

- m : number of approximation parameters.
- d : input dimension.
- The exponent deteriorates as d grows.

Curse of dimensionality

For a fixed accuracy, classical methods may require a number of parameters exponential in d .

Architectures, spaces and norms

Architecture $\rightarrow$ function space $\rightarrow$ complexity norm

Model	Finite hypothesis space	Natural space	Complexity
Random features	$\frac{1}{m} \sum_{j=1}^m a_j \phi(x; w_j^0)$	RKHS H_k	$\ \cdot\ _{H_k}$
Two-layer ReLU NN	$\frac{1}{m} \sum_{j=1}^m a_j \sigma(w_j^T x)$	Barron space B	$\ \cdot\ _B$
Residual networks	$z_{l+1} = z_l + \frac{1}{L} U_l \sigma(W_l z_l)$	Flow-induced spaces	flow norm
Multi-layer NN	$\sum_{i_L} a_{i_L}^L \sigma\left(\sum_{i_{L-1}} a_{i_L i_{L-1}}^{L-1} \sigma(\dots)\right)$	Tree-like spaces	path norm

Main idea

Each architecture has a natural function space.

The associated norm measures how complex a target function is for that architecture.

What does the norm do?

The goal is not just to find a norm, but to identify the right function space.

Function space

The space tells us **which targets** are naturally represented by the architecture.

$$f^* \in \mathcal{F}_*$$

Complexity norm

The norm tells us **how hard** the target is inside that space.

$$\|f^*\|_* < \infty$$

Analogy

FEM, PDE approximation	$\longleftrightarrow$	Sobolev spaces, e.g. H^1
Two-layer ReLU networks	$\longleftrightarrow$	Barron space

Random feature model

A two-layer network with activation σ has the form

$$f_m(x) = \frac{1}{m} \sum_{j=1}^m a_j \sigma(w_j^T x).$$

In the random feature model,

$$f_m(x) = \frac{1}{m} \sum_{j=1}^m a_j \phi(x; w_j^0),$$

where w_j^0 are sampled randomly and then frozen.

- Only the output coefficients a_j are trained.
- Hidden features are fixed.
- The model becomes linear in the trainable parameters.

Why useful?

Random features turn a neural-network-like model into linear regression on fixed nonlinear features.

Random features and RKHS

Let $\phi(x; w)$ be a feature map, for example

$$\phi(x; w) = \sigma(w^T x).$$

If $w \sim \pi_0$, define the kernel

$$k(x, x') = \mathbb{E}_{w \sim \pi_0} [\phi(x; w) \phi(x'; w)].$$

This kernel induces a reproducing kernel Hilbert space H_k .

Functions in H_k can be represented as

$$f(x) = \int a(w) \phi(x; w) d\pi_0(w), \quad \|a\|_{L^2(\pi_0)} < \infty.$$

Interpretation

The kernel describes the infinite-width feature model; random features approximate it by sampling finitely many features.

Two-layer ReLU networks and Barron space

For a two-layer ReLU network,

$$f_m(x) = \frac{1}{m} \sum_{j=1}^m a_j \sigma(w_j^T x), \quad \sigma(z) = \max(z, 0).$$

The infinite-width analogue is

$$f(x) = \mathbb{E}_\rho [a \sigma(w^T x)],$$

where ρ is a probability distribution over neuron parameters (a, w) .

Barron norm

$$\|f\|_{B_p} = \inf_{\rho \in P_f} \left(\mathbb{E}_\rho [|a|^p \|w\|_1^p] \right)^{1/p},$$

Barron space

$$B = \{f : \|f\|_B < \infty\}.$$

Residual and multi-layer networks: broader picture

Residual networks

A residual network has the form

$$z_{l+1} = z_l + \frac{1}{L} U_l \sigma(W_l z_l).$$

As $L \rightarrow \infty$, it resembles an ODE:

$$\dot{z}(t) = \mathbb{E}_{\rho_t}[U \sigma(Wz(t))].$$

Natural spaces: **flow-induced spaces**.

Complexity: **flow norms**.

Multi-layer networks

Represented by compositions of layers

$$\sum_{i_L} a_{i_L}^L \sigma \left(\sum_{i_{L-1}} a_{i_L i_{L-1}}^{L-1} \sigma(\dots) \right)$$

Natural spaces:

tree-like spaces W_L .

Complexity: **path-like norms**.

The theory is less clean than for two-layer networks.

Approximation theory

Direct and inverse approximation theorems

For a given architecture, H_m is the class of finite networks with width m .

Which targets are efficiently approximated as $m \rightarrow \infty$?

Direct approximation theorem

If f^* belongs to the associated function space $\mathcal{F}_*$, then

$$\inf_{f \in H_m} \|f - f^*\|_{L^2(P)}^2 \lesssim \frac{\|f^*\|_*^2}{m}.$$

Inverse approximation theorem

If f^* can be approximated efficiently by functions in H_m , with uniformly bounded complexity, then

$$\|f^*\|_* < \infty.$$

Thus f^* belongs to the associated function space $\mathcal{F}_*$.

Direct and inverse approximation theorems

Direct theorem

$$f^* \in \mathcal{F}_*, \quad \|f^*\|_* < \infty$$



efficient approximation by H_m .

Inverse theorem

efficient approximation by H_m



$$f^* \in \mathcal{F}_*, \quad \|f^*\|_* < \infty.$$

Interpretation

Together, the two directions justify $\mathcal{F}_*$ as the natural function space associated with the architecture.

Approximation results by model

Model	Natural space	Approximation statement
Random features	RKHS H_k	If $f^* \in H_k$, $\mathbb{E}_{W_0} \left\ f_m - f^* \right\ _{L^2(P)}^2 \lesssim \frac{\ f^*\ _{H_k}^2}{m}.$
Two-layer ReLU NN	Barron space B	If $f^* \in B$, $\left\ f_m - f^* \right\ _{L^2(P)}^2 \lesssim \frac{\ f^*\ _B^2}{m}.$
Residual networks	Flow-induced space $\widetilde{D}_2$	If $f^* \in \widetilde{D}_2$, $\left\ f^* - f_L(\cdot; \Theta) \right\ _{L^2(P)}^2 \lesssim \frac{\ f^*\ _{\widetilde{D}_2}^2}{L^{1-\delta}}.$
Multi-layer NN	Tree-like spaces W_L	If $f^* \in W_L$, $\left\ f_m - f^* \right\ _{L^2(P)}^2 \lesssim \frac{4^L \ f^*\ _{W_L}^2}{m}.$

Limitation of the Monte Carlo approximation argument

For Barron functions, one common proof uses the infinite representation

$$f(x) = \mathbb{E}_{\rho} [a\sigma(w^T x)] .$$

Then a finite network is obtained by sampling neuron parameters:

$$f_m(x) = \frac{1}{m} \sum_{j=1}^m a_j \sigma(w_j^T x), \quad (a_j, w_j) \sim \rho.$$

This gives a typical random-sampling error:

$$\mathbb{E}_{(a_j, w_j) \sim \rho} \|f - f_m\|_{L^2(P)}^2 \lesssim \frac{\|f\|_B^2}{m}.$$

Open issue

Training does not sample parameters randomly; it optimizes them. The true best-approximation problem is

$$\inf_{\theta} \|f - f_{\theta}\|_{L^2(P)}^2 .$$

Estimation and generalization

Generalization gap

For the trained model $\hat{f}$,

$$R(\hat{f}) = \mathbb{E}_{x \sim P} \left[(\hat{f}(x) - f^*(x))^2 \right],$$

while

$$\hat{R}_n(\hat{f}) = \frac{1}{n} \sum_{i=1}^n (\hat{f}(x_i) - f^*(x_i))^2.$$

The generalization gap is

$$\left| R(\hat{f}) - \hat{R}_n(\hat{f}) \right|.$$

Since $\hat{f}$ depends on the same training data, a standard strategy is

$$\left| R(\hat{f}) - \hat{R}_n(\hat{f}) \right| \leq \sup_{f \in \mathcal{F}} \left| R(f) - \hat{R}_n(f) \right|.$$

Question

How statistically complex is the class $\mathcal{F}$?

Rademacher complexity

For a function class F and a sample $S = (x_1, \dots, x_n)$,

$$\text{Rad}_S(F) = \frac{1}{n} \mathbb{E}_{\xi} \left[\sup_{h \in F} \sum_{i=1}^n \xi_i h(x_i) \right],$$

where $\xi_i \in \{-1, +1\}$ are independent random signs.

Interpretation

$\text{Rad}_S(F)$ measures how well functions in F can fit random noise on the sample S .

Connection to generalization

Small Rademacher complexity implies a smaller worst-case generalization gap.

Connecting approximation and estimation

The population risk $R(\hat{f})$ is controlled by two mechanisms.

1. **Approximation:** if $f^* \in \mathcal{F}_*$, then there exists $f_m \in H_m$ such that

$$R(f_m) = \|f_m - f^*\|_{L^2(P)}^2 \lesssim \frac{\|f^*\|_*^2}{m}.$$

2. **Estimation:** if training is restricted to

$$F_Q = \{f : \|f\|_* \leq Q\},$$

then Rademacher complexity controls

$$\sup_{f \in F_Q} |R(f) - \hat{R}_n(f)| \lesssim \text{Rad}_S(F_Q).$$

For the ideal case,

$$\text{Rad}_S(F_Q) \lesssim \frac{Q}{\sqrt{n}}.$$

Approximation plus estimation

Combining the two mechanisms gives a typical a priori estimate:

$$R(\hat{f}) \lesssim \underbrace{\frac{\|f^*\|_*^2}{m}}_{\text{approximation error}} + \underbrace{\frac{\|f^*\|_*}{\sqrt{n}}}_{\text{estimation error}}.$$

For two-layer ReLU networks and Barron space:

$$R(\hat{f}) \lesssim \frac{\|f^*\|_B^2}{m} + \|f^*\|_B \sqrt{\frac{\log d}{n}}.$$

Main message

A target function is learnable by an architecture when it has low complexity in that architecture's natural function space.

A priori vs a posteriori estimates

A priori

Depends on the unknown target:

$$R(\hat{f}) \lesssim \frac{\|f^*\|_*^2}{m} + \frac{\|f^*\|_*}{\sqrt{n}}.$$

Useful theoretically, but $\|f^*\|_*$ is usually unknown.

A posteriori

Depends on the trained model:

$$R(\hat{\theta}_n) - \hat{R}_n(\hat{\theta}_n) \lesssim \frac{\|\hat{\theta}_n\|}{\sqrt{n}}.$$

Computable, but often vacuous because the norm can be very large.

Numerical analysis analogy

In FEM, a posteriori estimates guide mesh refinement. For neural networks, analogous adaptive theory remains largely open.

Generalization results by model

Model	Bounded class	Generalization control
Random features	$F_Q = \{f : \ f\ _{H_k} \leq Q\}$	$\text{Rad}_S(F_Q) \lesssim \frac{Q}{\sqrt{n}}$
Two-layer ReLU NN	$F_Q = \{f : \ f\ _B \leq Q\}$	$\text{Rad}_S(F_Q) \lesssim Q \sqrt{\frac{\log d}{n}}$
Residual networks	$\widehat{D}_2^Q = \{f : \ f\ _{\widehat{D}_2} \leq Q\}$	$\text{Rad}_S(\widehat{D}_2^Q) \lesssim Q \sqrt{\frac{\log d}{n}}$
Multi-layer NN	$F_Q = \{f : \ f\ _{W_L} \leq Q\}$	$\text{Rad}_S(F_Q) \lesssim 2^{L+1} Q \sqrt{\frac{\log d}{n}}$

Open issue: sharper generalization estimates

Standard Rademacher bounds are global:

$$\left| R(\hat{f}) - \widehat{R}_n(\hat{f}) \right| \leq \sup_{f \in F_Q} \left| R(f) - \widehat{R}_n(f) \right|.$$

Limitation

The right-hand side looks at the worst function in the whole class F_Q .

However, after training, $\hat{f}$ should have small pointwise loss:

$$\ell_{\hat{f}}(x) = (\hat{f}(x) - f^*(x))^2.$$

Message

A sharper theory should exploit that training selects functions with small loss, not arbitrary functions in the complexity ball.

From approximation theory to training dynamics

From approximation theory to computation

In the previous sections we established, for each function space, two theoretical guarantees:

Approximation

If f^* has finite complexity in the associated space $\mathcal{F}_*$, then good functions *exist* in H_m :

$$\inf_{f \in H_m} \|f - f^*\|_{L^2(P)}^2 \lesssim \frac{\|f^*\|_*^2}{m}.$$

Generalization gap

Restricting to a norm ball controls the gap between training and test error:

$$\sup_{f \in F_Q} |R(f) - \hat{R}_n(f)| \lesssim \frac{Q}{\sqrt{n}}.$$

The missing piece

These are *existence* results. They do not tell us whether gradient descent actually **finds** such solutions, or whether those solutions **generalize** in practice.

Training dynamics: main questions

Optimization

Does gradient descent converge to a good solution? How fast?

Answer depends on the model and the scaling regime.

Generalization and implicit regularization

Does the solution found by the optimizer generalize to test data?

In the over-parametrized regime many global minima exist with zero training error. The optimizer *selects* among them — this selection acts as an **implicit regularizer**, preferring solutions with better generalization.

Key point

The optimizer is not just a numerical detail. It defines which solution is returned.

Overview: what is covered in this section

Model	Convergence	Generalization	Experiment Source
Random feature model	✓	✓	[2]
Two-layer NN (mean-field)	✓	–	[1]
Two-layer NN (conventional)	✓	✓	[3]
Adam	✓	✓	[4]
Global minima: GD vs. SGD	–	✓	[5]

Reading guide

Convergence: does the training loss reach a (global) minimum, and how fast?

Generalization: does the trained model generalize well, and which minimum is selected?

Optimization algorithm

Only Full Gradient Descent is covered for each model.

Random feature model: definition and GD solution

Model. Given frozen random features $\{w_j\}_{j=1}^m \sim \pi$, the model is

$$\hat{f}(x; a) = \sum_{k=1}^m a_k \phi(x; w_k), \quad a \in \mathbb{R}^m.$$

Training solves the linear least squares problem $\min_{a \in \mathbb{R}^m} \frac{1}{2n} \|\Phi a - y\|^2$ with $\Phi_{ij} = \phi(x_i; w_j)$.

Gram matrix. Φ encodes the inner products between features at training points. Its singular values $\{\lambda_i\}$ control stability:

$$\Phi = U \Sigma V^T, \quad \Sigma = \text{diag}(\lambda_1, \dots, \lambda_{\min(n,m)}).$$

Closed-form GD path (initialized at $a = 0$):

$$\hat{f}_t(x) = \phi(x; B)^T a(t) = \sum_{i: \lambda_i > 0} \frac{1 - e^{-\lambda_i^2 t / (mn)}}{\lambda_i} (u_i^T y) (v_i^T \phi(x; B)).$$

Key insight

Each SVD mode enters the solution at rate λ_i^2 . When $m \approx n$, very small λ_i appear, causing **double descent** and **slow deterioration**.

Random feature model: results and replication

Setup

Data

MNIST, $n = 500$

Model

RF with ReLU features

Varied

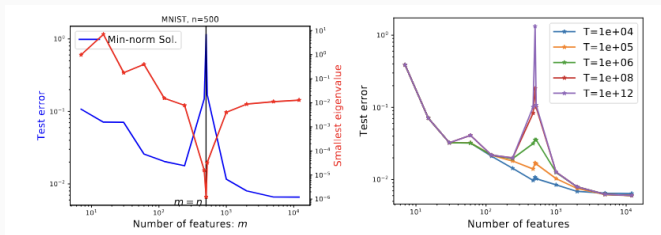
$m \in [10^1, 10^4]$

Optimizer

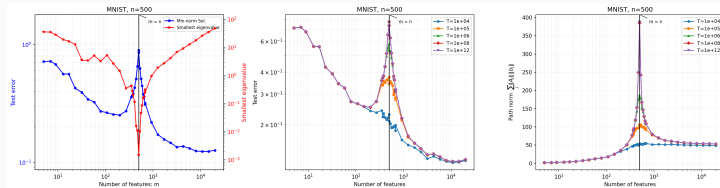
GD from $a = 0$,
min-norm solution

Question

Does test error peak at $m = n$
and deteriorate only after
many steps?



Paper (Fig. 6): test error and smallest eigenvalue vs. m ; slow deterioration regimes.



Our replication.

Random feature model: paper results and replication

Setup

Data

MNIST, $n = 500$

Model

RF with ReLU features,
 B fixed random

Threshold

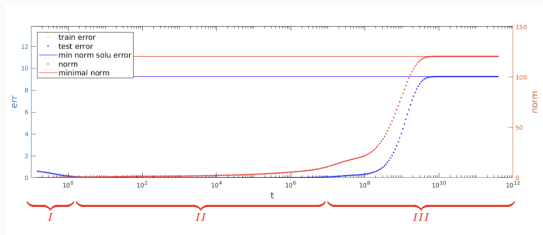
$m = n = 500$

X-axis

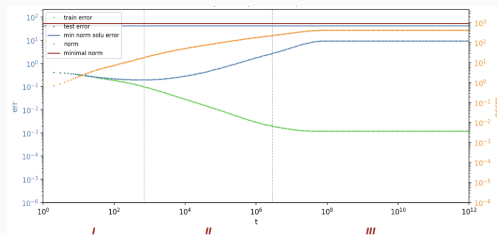
GD iterations T
 $T \in [1, 10^{12}]$ (log)

Question

Do the three phases — fast drop, stable plateau, late deterioration — appear as predicted by the spectral structure of Φ ?



Paper (Fig. 7): test error (blue) and smallest Gram eigenvalue (red)



Our replication.

Two-layer NN (mean-field scaling): model and convergence

Model.

$$f_m(x) = \frac{1}{m} \sum_{j=1}^m a_j \sigma(w_j^T x), \quad u_j = (a_j, w_j).$$

As $m \rightarrow \infty$, tracking the empirical measure $\rho_t = \frac{1}{m} \sum_j \delta_{u_j(t)}$ yields the *mean-field (Wasserstein gradient flow)* equation:

$$\partial_t \rho = \nabla(\rho \nabla V), \quad V = \frac{\widehat{\delta R_{nn}}}{\delta \rho}.$$

Theorem 36 [Chizat–Bach, 2018]

Let $\{\rho_t\}$ be a solution of the Wasserstein gradient flow with initial distribution ρ_0 on the cone $\{|a|^2 \leq |w|^2\}$ and *omni-directional* support (every open cone has positive measure). Then the following are equivalent:

1. The velocity potentials $\frac{\delta R}{\delta \rho}(\rho_t, \cdot)$ converge to a unique limit.
2. $R(\rho_t)$ decays to the global infimum $= 0$.

If either holds, ρ_t converges in Wasserstein distance to a global minimizer.

Two-layer NN (mean-field): paper results and replication

Setup

Data

$x \sim \text{Uniform}([0, 1]^d)$, $n = 1000$
 $d \in \{30, 100, 250\}$

Model

Mean-field two-layer NN:

Optimizer

Mean-field GD with step size η

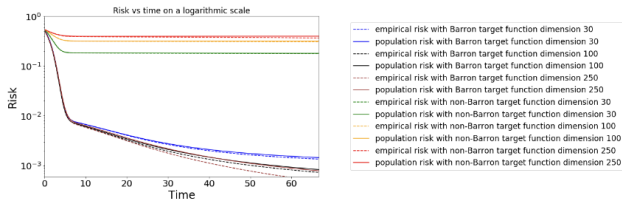
Targets

Barron: $\sigma(x_1)$, $\|f^*\|_B = 1$

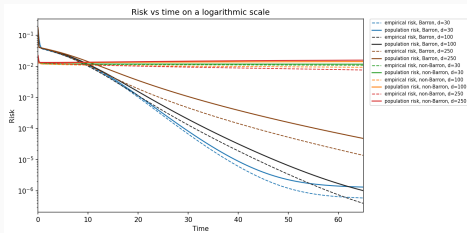
Non-Barron: $\sigma\left(\frac{\sum_k x_k - d/2}{\sqrt{d}}\right)$,
 $\|f^*\|_B = \sqrt{d}$

Question

Is convergence dimension-independent for the Barron target, and does it slow with d for the non-Barron target?



Paper (Fig. 2): convergence of Barron & non-Barron targets across dimensions.



Our replication.

Two-layer NN (conventional scaling): model

Model. $f_m(x; a, B) = \sum_{j=1}^m a_j \sigma(b_j^T x) = a^T \sigma(Bx)$ with $a_j(0) \sim \mathcal{N}(0, \beta^2)$, $b_j(0) \sim \mathcal{N}(0, I/d)$.

Theorem 39 [Du et al., 2019] — convergence

Let $\lambda_n = \lambda_{\min}(K)$ where $K_{ij} = \frac{1}{n} \mathbb{E}_{b \sim \pi_0} [\sigma(b^T x_i) \sigma(b^T x_j)]$ and assume $\beta = 0$. For $m \gtrsim n^2 \lambda_n^{-4} \delta^{-1} \log(n^2 \delta^{-1})$, with prob. $\geq 1 - 6\delta$:

$$\widehat{R}_{nn}(a(t), B(t)) \leq e^{-m \lambda_n t} \widehat{R}_{nn}(a(0), B(0)).$$

Exponential convergence to the empirical minimum.

Theorem 40 [E, Ma, Wu, 2020] — generalization

Under the same conditions, the full GD path $f_m(\cdot; a(t), B(t))$ stays uniformly close to the *random feature* path $f_m(\cdot; \tilde{a}(t), B_0)$:

$$\sup_{x \in S^{d-1}} |f_m(x; a(t), B(t)) - f_m(x; \tilde{a}(t), B_0)| \lesssim \frac{(1 + \sqrt{\ln(1/\delta)})^2}{\lambda_n \sqrt{m}}.$$

In the highly over-parametrized regime there is no implicit regularization: the NN generalizes no better than its frozen-feature counterpart.

Two-layer NN (conventional scaling): paper results and replication

Setup

Data

$x \sim \text{Uniform}([0, 1]^d)$
Target: $f^*(x) = \sigma(x_1)$

Mild Regime

$m = 30, n = 200, d = 19$

Model

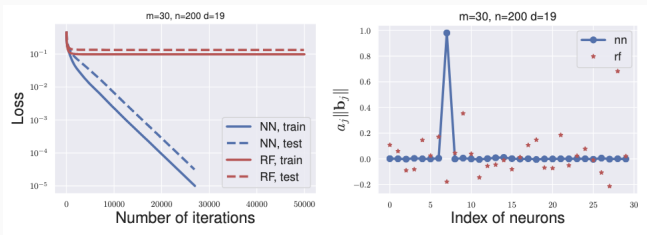
NN: GD on both a_j and B
RF: B frozen at init

Optimizer

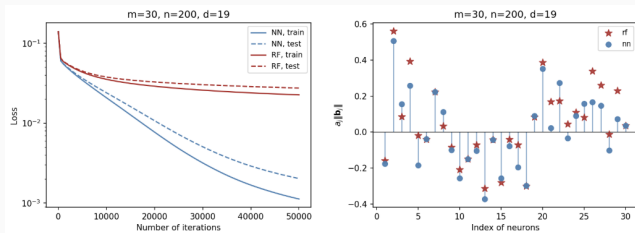
Full-batch GD, $\eta = 10^{-3}$
50 000 iterations

Question

Does the NN outperform RF by learning to activate only a few neurons, while RF saturates early?



Paper (Fig. 3, top): loss curves (left) and per-neuron coefficients $a_j \|b_j\|$ (right)



Our replication.

Two-layer NN (conventional scaling): paper results and replication

Setup

Data

$x \sim \text{Uniform}([0, 1]^d)$
Target: $f^*(x) = \sigma(x_1)$

High Regimes

$m = 2000, n = 200, d = 20$

Model

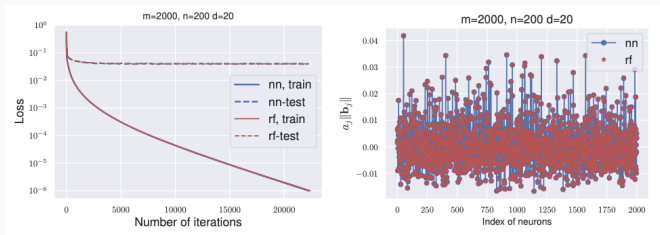
NN: GD on both a_j and B
RF: B frozen at init

Optimizer

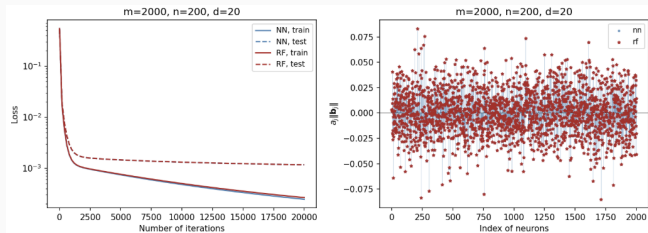
Full-batch GD, $\eta = 10^{-3}$

Question

Does the NN outperform RF by activating a few neurons (mild regime), and does this advantage vanish at large m (high regime)?



Paper (Fig. 3): Left: loss curves. Right: per-neuron coefficients $a_j ||b_j||$.



Our replication.

Two-layer NN (conventional scaling): paper results and replication

Setup

Data

$x \sim \mathcal{N}(0, I_d/d)$, $d = 20$, $n = 200$

Teacher: $NN_{m=5}^*$ with $\text{std}(y_{\text{train}}) = 1$

Models

NN: GD on a_j and B

RF: B fixed, a via GD

Sweep

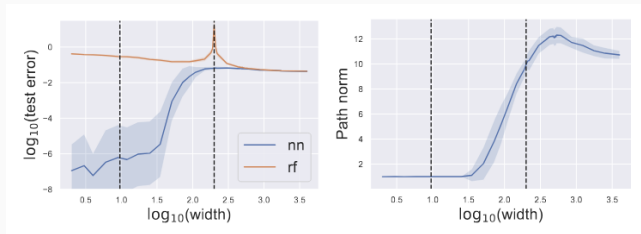
$\log_{10}(m) \in [0.5, 3.5]$

Optimizer

Full-batch GD, $\eta = 10^{-3}$

Question

Does the NN test error spike near $m = n$ and does the path norm surge at the same threshold, explaining the transition to RF-like behaviour?



Paper (Fig. 5): **Left:** $\log_{10}(\text{test error})$ vs $\log_{10}(m)$ for NN and RF models. **Right:** path norm $\sum_j |a_j| \|b_j\|$ vs $\log_{10}(m)$. Dashed lines at $m = n/(d+1)$ and $m = n$.

Adam: algorithm and hyperparameter regimes

Update rules (bias-corrected):

$$v_{t+1} = \alpha v_t + (1 - \alpha)(\nabla f(x_t))^2, \quad m_{t+1} = \beta m_t + (1 - \beta)\nabla f(x_t),$$
$$x_{t+1} = x_t - \eta \frac{m_{t+1}/(1 - \beta^{t+1})}{\sqrt{v_{t+1}/(1 - \alpha^{t+1})} + \varepsilon}.$$

Setting $\alpha = 1 - a\eta$, $\beta = 1 - b\eta$ and letting $\eta \rightarrow 0$ yields a continuous limit that reveals sensitivity to the ratio a/b . Three qualitatively distinct regimes arise in the (a, b) hyperparameter space:

Spike ($b \gg a$)

Large sudden spikes in the loss; unstable training.

Oscillation ($a \approx b$)

Fast, small oscillations; small, stable loss.

Practical target.

Divergence ($a \gg b$)

Loss grows without bound; training fails.

Fast initial convergence is partly explained by the signGD limit ($\varepsilon \rightarrow 0$, $\eta \rightarrow 0$): for PL-condition objectives, signGD reaches the minimum in finite time.

Adam: paper results and replication

Setup

Data

CIFAR-10, 2 classes

1000 samples per class

Network

FC: $256 \rightarrow 128 \rightarrow 64$

ReLU activations

MSE loss (one-hot)

Adam

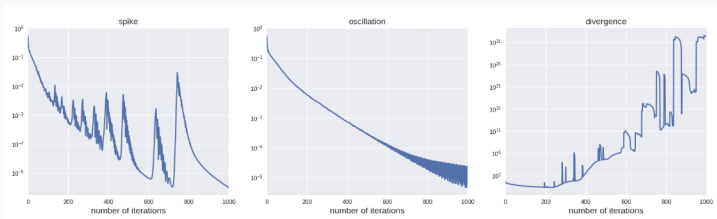
$\eta = 10^{-3}$, 1000 iters, Full batch

$\alpha = 1 - a\eta$

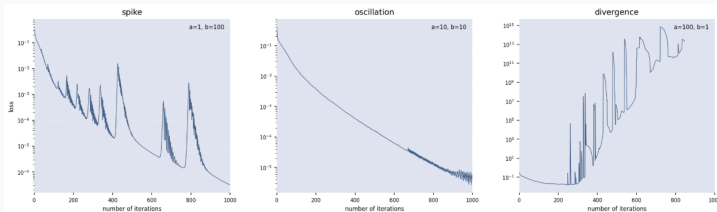
$\beta = 1 - b\eta$

Question

Do the three qualitative regimes — spike, oscillation, divergence — arise purely from the ratio a/b , with $a \approx b$ giving the only stable outcome?



Paper (Fig. 10): loss curves for spike ($a = 1, b = 100$), oscillation ($a = 10, b = 10$) and divergence ($a = 100, b = 1$).



Our replication.

Adam: paper results and replication

Setup

Data

CIFAR-10, 2 classes

500 samples per class

Network

FC: 3072→256→256→128→2

ReLU, MSE loss

Adam

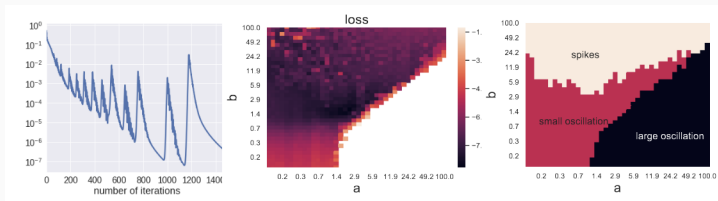
$\eta = 10^{-3}$, 1500 iters, Full batch

Grid

$a, b \in [0.1, 100]$ (log)

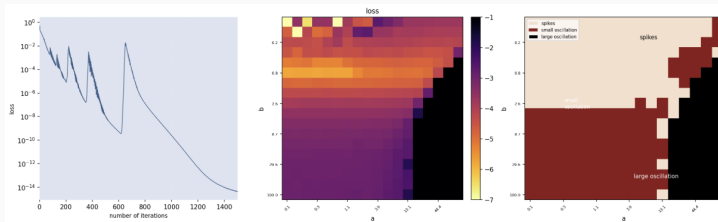
Question

Does the oscillation regime ($a \approx b$) form a stable band in (a, b) space, and do default Adam parameters fall in the spike regime?



Paper (Fig. 11): Left: training curve at default $(\alpha, \beta) = (0.9, 0.999)$ — spike regime.

Middle: heat map of avg. training loss over (a, b) grid. Right: behaviour classification.



Our replication.

Global minima selection: GD vs. SGD

In the over-parametrized regime, the set of global minimizers of $\hat{R}_{nn}$ is a manifold of dimension $m - n$ [6]. Different algorithms select different points on this manifold.

Dynamic stability perspective [7]

A global minimum is *dynamically stable* for an optimizer if small perturbations do not cause the iterate to escape. The set of stable minima differs between GD and SGD.

For a 1-D model $f(x) = \frac{1}{2n} \sum_i a_i x^2$, stability requires:

$$\text{GD: } |1 - \eta a| \leq 1, \quad \text{SGD (additionally): } s \leq \frac{2}{\eta},$$

where $a = \frac{1}{n} \sum_i a_i$ is the **sharpness** and $s = \sqrt{\frac{1}{n} \sum_i a_i^2 - a^2}$ is the **non-uniformity**.

- SGD imposes a stricter stability condition through non-uniformity.
- Hence SGD selects **flatter**, more uniform minima than GD.
- This is the mathematical basis for the well-known empirical observation that SGD generalizes better than full-batch GD.

Global minima selection: GD vs. SGD — paper results and replication

Setup

Data

FashionMNIST

1000 correct labels + 200 random labels

Network

LeNet-like (~11k params)

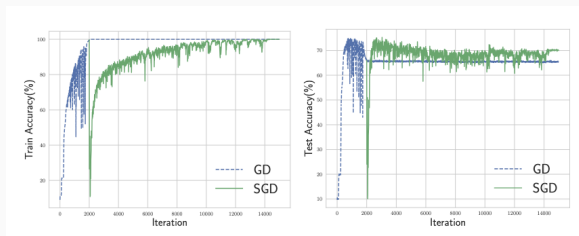
MSE loss

Protocol

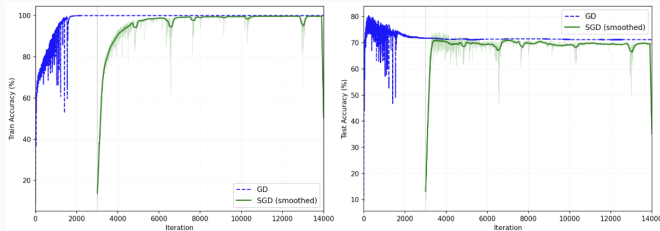
1. GD for 3000 iters
 2. Switch to SGD
- $\eta = 0.07$ throughout
14000 iters total

Question

Does SGD escape the sharp minimum θ_{GD}^* found by GD and converge to a flatter minimum with better test accuracy?



Paper (Fig. 8): escape phenomenon. **Left:** training accuracy. **Right:** test accuracy.



Our replication.

Global minima selection: GD vs. SGD — paper results and replication

Setup

Data FashionMNIST

Network VGG-type with MSE Loss

Optimizers GD (full batch), SGD with $B \in \{4, 10, 25\}$

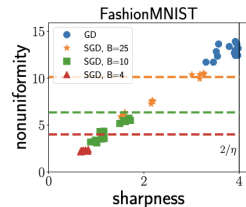
Sharpness $a = \lambda_{\max}(\nabla^2 L)$ via power iteration

Non-uniformity $s = \sqrt{\frac{1}{n} \sum_i a_i^2 - a^2}$

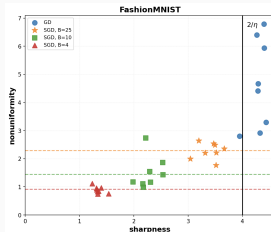
Theory bound SGD stable iff $s \leq 2/\eta$, GD clusters near $a = 2/\eta$

Question

Do SGD minima satisfy the predicted bound $s \leq 2/\eta$, and is the bound tight across all batch sizes?



Paper (Fig. 9): sharpness–non-uniformity diagram.



Our replication.

Training dynamics: key takeaways

Random feature model

Small eigenvalues at $m \approx n$ activate late but blow up as $1/\lambda_i$ — causing **slow deterioration**. Early stopping implicitly regularizes.

Mean-field scaling

Wasserstein gradient flow converges globally under omni-directional initialization.

Dimension-free for Barron targets; **slow** otherwise.

Conventional scaling

Exponential convergence, but the network stays $O(1/\sqrt{m})$ -close to its **frozen-feature** counterpart. No feature learning at high over-parametrization.

Adam

Three regimes from the ratio a/b : **spike**, **oscillation** (practical target), **divergence**. Default parameters fall in the spike regime.

GD vs. SGD

SGD's extra stability condition on non-uniformity s selects **flatter** minima — the mathematical basis for better generalization.

Common thread

The optimizer is not a numerical detail: it determines **which** solution is returned and **how well** it generalizes.

Conclusions

What we know

- Each architecture has a **natural function space** and complexity norm.
- Barron functions admit **dimension-free** Monte Carlo approximation rates.
- **Rademacher complexity** controls generalization for bounded-norm classes.
- Over-parametrization guarantees convergence but collapses to **random features**.
- Optimizers implicitly regularize by **selecting** among global minima.

What we do not know

- Sharper approximation beyond **Monte Carlo sampling**.
- How to exploit small **pointwise loss** in generalization bounds.
- Whether current spaces fit **realistic deep networks**.
- Natural spaces for **CNNs, DenseNets**, modern architectures.
- A complete theory of **implicit regularization** for GD, SGD, Adam.

Takeaway

The framework exists. The sharp theory remains open.

Final takeaway

What the paper gives us

A mathematical language to discuss neural networks as approximation schemes:

architecture $\longrightarrow$ function space $\longrightarrow$ complexity norm.

- **Approximation:** solutions exist when the target has low complexity in the right space.
- **Generalization:** Rademacher complexity controls finite-sample error.
- **Training dynamics:** optimization selects one solution among many.

Main lesson

The success of neural networks cannot be explained by approximation alone:

good architecture + controlled complexity + favorable training dynamics

Thank you

Questions?



GitHub replication project

- [1] W. E, C. Ma, S. Wojtowytsch, and L. Wu, ***Towards a Mathematical Understanding of Neural Network-Based Machine Learning: What we know and what we don't***, en, arXiv:2009.10713 [cs], Dec. 2020. DOI: 10.48550/arXiv.2009.10713. Accessed: May 6, 2026. [Online]. Available: <http://arxiv.org/abs/2009.10713>.
- [2] C. Ma, L. Wu, and W. E, **“The Slow Deterioration of the Generalization Error of the Random Feature Model,”** en, in *Proceedings of The First Mathematical and Scientific Machine Learning Conference*, PMLR, Aug. 2020, pp. 373–389. Accessed: Jun. 1, 2026. [Online]. Available: <https://proceedings.mlr.press/v107/ma20a.html>.
- [3] C. Ma and L. Wu, **“The Quenching-Activation Behavior of the Gradient Descent Dynamics for Two-layer Neural Network Models,”** en,
- [4] C. Ma and L. Wu, **“A Qualitative Study of the Dynamic Behavior for Adaptive Gradient Algorithms,”** en,

- [5] Z. Zhu, J. Wu, B. Yu, L. Wu, and J. Ma, ***The Anisotropic Noise in Stochastic Gradient Descent: Its Behavior of Escaping from Sharp Minima and Regularization Effects***, arXiv:1803.00195 [stat.ML], Jun. 2019. DOI: 10.48550/arXiv.1803.00195. Accessed: May 31, 2026. [Online]. Available: <http://arxiv.org/abs/1803.00195>.
- [6] Y. Cooper, ***The loss landscape of overparameterized neural networks***, <https://arxiv.org/abs/1804.10200v1>, Apr. 2018. Accessed: Jun. 1, 2026.
- [7] L. Wu, C. Ma, and W. E, **“How SGD Selects the Global Minima in Over-parameterized Learning: A Dynamical Stability Perspective,”** in *Advances in Neural Information Processing Systems*, vol. 31, Curran Associates, Inc., 2018. Accessed: Jun. 1, 2026.

Thank you

Questions?



GitHub replication project